Application Assistant

A Discord bot that tailors your CV, cover letter and interview prep to each job offer

Pollux GRONIER · October 2026

Tool n8n Cloud **Channel** Discord (slash commands) **Database** n8n Data Tables **AI** Gemini 3.5 Flash via OpenRouter **Output** Google Docs + PDF **Level** 3

The need, unchanged from the scoping. Students who apply to many internships rewrite their CV and cover letter by hand for every posting. Generic AI chats help, but you have to paste your background, the offer and the prompt again every time.

What was built. A private Discord bot driven by four slash commands. You upload your CV once (`/cv`), send any job link (`/job`), answer one targeted question about your missing skills (`/answer`), and get back a tailored CV as a PDF, a cover letter and an interview cheat sheet, all saved in a Google Drive folder. Everything you tell the bot is remembered for the next applications.

Status (final version, 22 October 2026). Every test from the scoping plan passes in the live system, re-run end to end on the final build through Discord with a real offer (section 6). None of the cut line had to be used: CV, letter *and* cheat sheet are all delivered. The tests also exposed ten problems, all fixed in the final build (section 7).

1 From the scoping to the build: what changed and why

The scoping deliverable was the contract. Most of it was built as written; four choices changed once I hit the real platforms.

Topic	Scoped	Built, and why
Trigger	Discord Trigger on every channel message	**Slash commands** received by an n8n **Webhook** (Discord "Interactions Endpoint URL"). n8n has no built-in trigger for Discord messages: that needs a permanent Gateway connection, which a cloud workflow cannot hold. Interactions are plain HTTP POSTs, so they fit a webhook, and they arrive already structured (file, url, text), so there is no guessing about the message type.
Routing	Switch on message type (PDF / URL / text)	Switch on the **command name** (`/cv`, `/job`, `/answer`); `/help` and unknown input get the usage message. Same idea, but deterministic.
AI provider	Groq / Gemini	**Gemini 3.5 Flash through OpenRouter**: one key, the model can be swapped in one field, fast and cheap enough for 5 calls per application.
Output	Formatted text in Discord	**Google Docs**: the CV is filled into my own fixed CV template, then exported to PDF and attached in Discord; letter and cheat sheet are Docs I can still edit. Long documents do not fit a 2,000-character Discord message.
Database	`facts` and `sessions`	Same two tables plus a third, `runs`, so the error workflow can find which user to warn (section 4).

Other design choices

- **AI reads, code decides.** The AI handles messy text (CV, offer, answer) and returns JSON; Code nodes do the comparisons, scores and formatting. Code is free, instant and gives the same result every time.
- **LLM Chains, not an AI Agent.** The steps are always the same and in the same order. A chain with a structured output parser gives typed JSON; an agent would choose its own steps, cost more calls and be less predictable.
- **Fill a template, don't generate a CV.** The AI writes five fields of my real CV; layout, dates and titles cannot break.
- **One question, not a questionnaire.** The bot asks about at most three missing items in one message, so an application stays three commands long.
- **Set nodes as contracts.** A Set node before each Execute Workflow decides exactly what the sub-workflow receives.

- **Temperature per task.** 0.1 for extraction (CV, offer, facts), 0.6 for the question and the CV wording, 0.8 for the letter.

2 Architecture overview

Five workflows: one router, two sub-workflows, one error workflow and a one-off setup. Discord talks to n8n over HTTP only; the AI, the database and Google are called from inside the workflows.

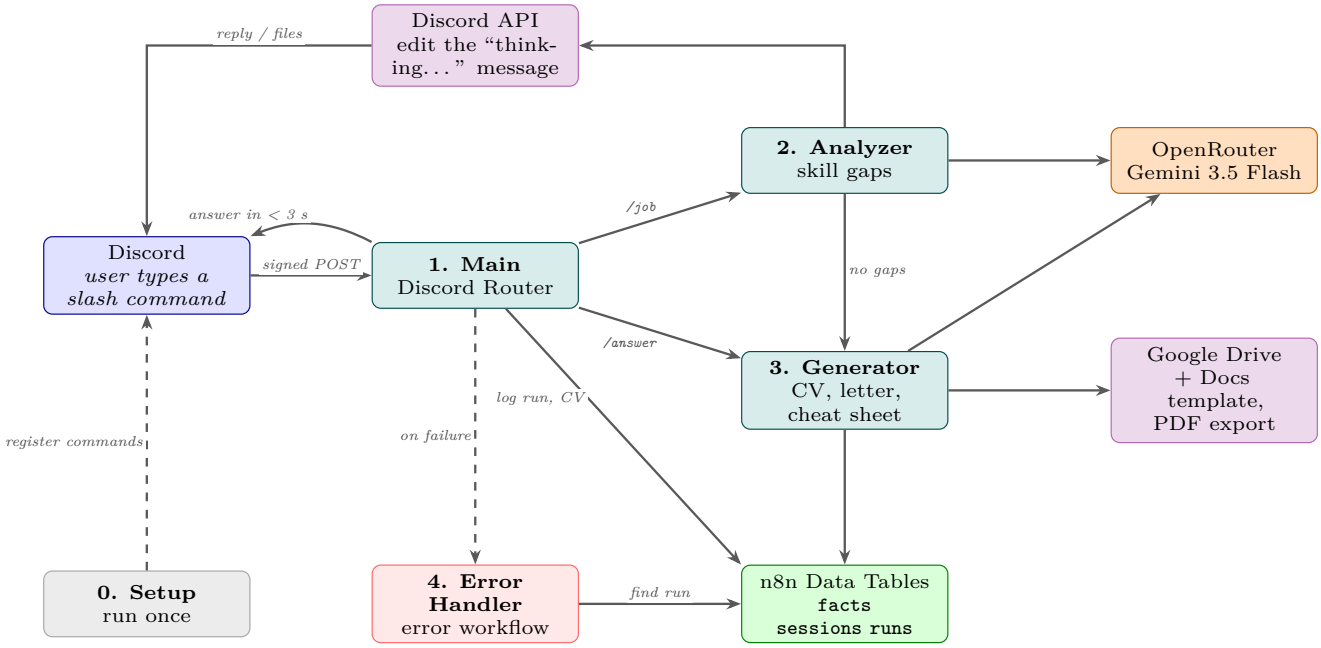


Figure 1: System overview. Solid arrows: normal flow. Dashed: one-off or failure paths.

2.1 The Discord contract (the key to the whole design)

1. Discord sends every slash command as an HTTP POST with an **Ed25519 signature**. Main checks it in a Code node with the app's public key and answers `401` if it is wrong. Discord refuses to save the endpoint URL unless this check works.
2. Discord needs an answer within **3 seconds**. Main answers at once: the usage text for `/help`, otherwise a "deferred" response that shows *"thinking..."*.
3. The real work then continues in the background. When it is done, the workflow **edits that message** through the Discord API (`PATCH /webhooks/{app}/{token}/messages/@original`). The interaction token stays valid for 15 minutes, which is plenty: a full generation takes about one minute.

Why: Answering fast and finishing later is the only way to run slow AI calls behind Discord's 3-second limit.

3 The five workflows, and when each one runs

Workflow	Started by	Job
0. Setup	Me, by hand, once	Registers `/help`, `/cv`, `/job`, `/answer` on my server through the Discord API. Re-run only if the commands change.
1. Main, Discord Router	Every slash command (Webhook)	Checks the signature, answers within 3 s, logs the run, routes the command. Handles `/cv` itself.
2. Analyzer	Main, on `/job` (Execute Workflow)	Gets the offer, extracts requirements, finds the gaps, asks one question, saves the session.
3. Generator	Main, on `/answer`, *or* right after the Analyzer when nothing is missing	Learns from the answer, writes the CV, letter and cheat sheet, sends the PDF.
4. Error Handler	n8n, whenever Main, Analyzer or Generator still fails after retries	Alerts me in the admin channel and tells the user something went wrong.

3.1 1. Main: receiving a command

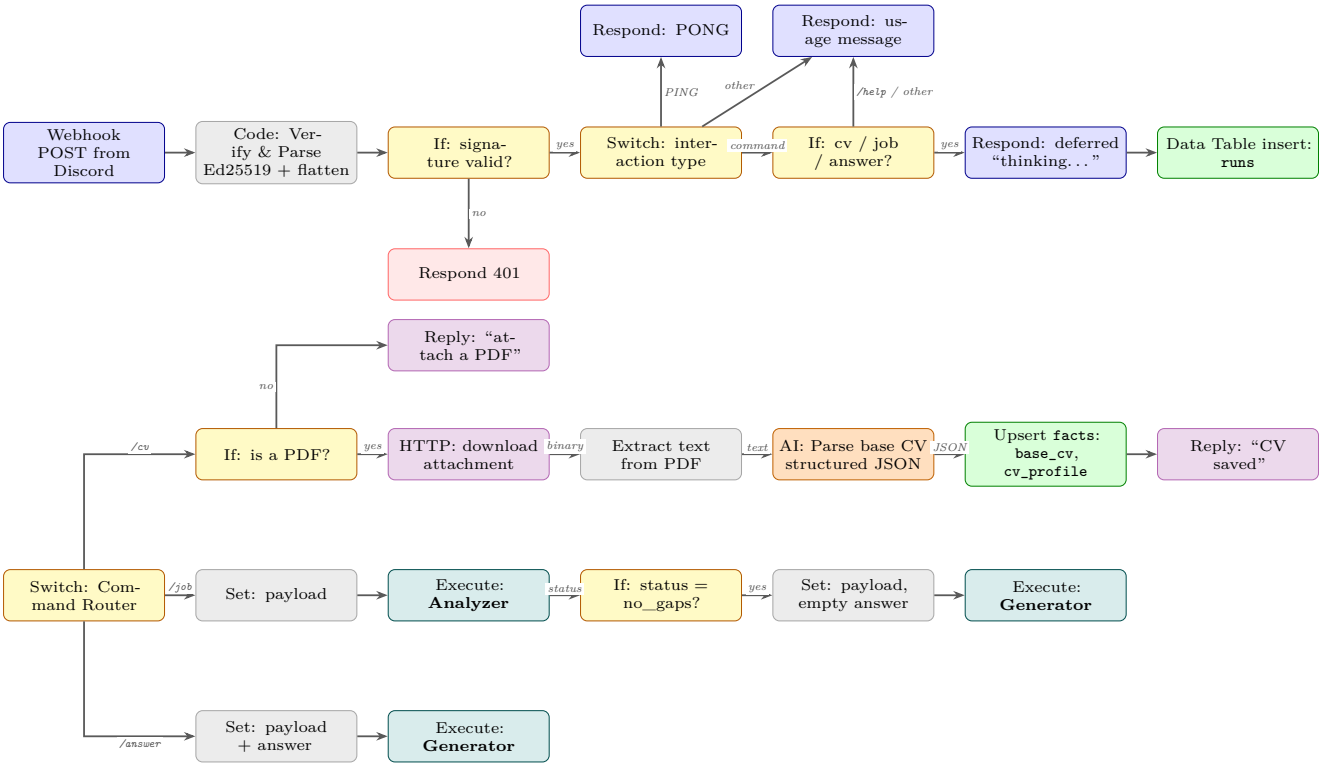


Figure 2: Main. Top: the fast part that must finish within 3 s. Bottom: the router behind it.

What travels: after Verify & Parse, every run carries one small object: `user_id`, `command`, `url`, `text`, `attachment` and the interaction `token` needed to reply. Sub-workflows receive exactly this object.

3.2 2. Analyzer: finding the skill gaps

How the gap analysis works. The AI only *reads* the offer: it returns 6 to 12 requirements as short keywords ("SQL", "Agile", "Stakeholder management") plus company, role and language. A Code node then compares each keyword with everything known about me (CV text, parsed profile, confirmed skills). Text is lower-cased and accents are removed, and only the *values* of the profile are searched, never JSON field names such as "tools". A requirement counts as matched if the whole phrase, or every significant word of it, appears as a **whole word**: "Git" does not match "github.com" and "Go" does not match "Google". The score is $10 \times \frac{\text{matched}}{\text{requirements}}$. The first three missing items become the question, and **all** missing items are saved, so the Generator can never claim any of them.

Why: the comparison is deterministic, free and explainable: I can show exactly why a skill counts as missing. An AI judge would be slower, cost a call, and give different answers for the same input.

How the page is read. Many job boards (LinkedIn, Indeed, Welcome to the Jungle, Lever) embed a schema.org `JobPosting` block. The cleaner takes that first, and only falls back to stripping the whole HTML. If the request

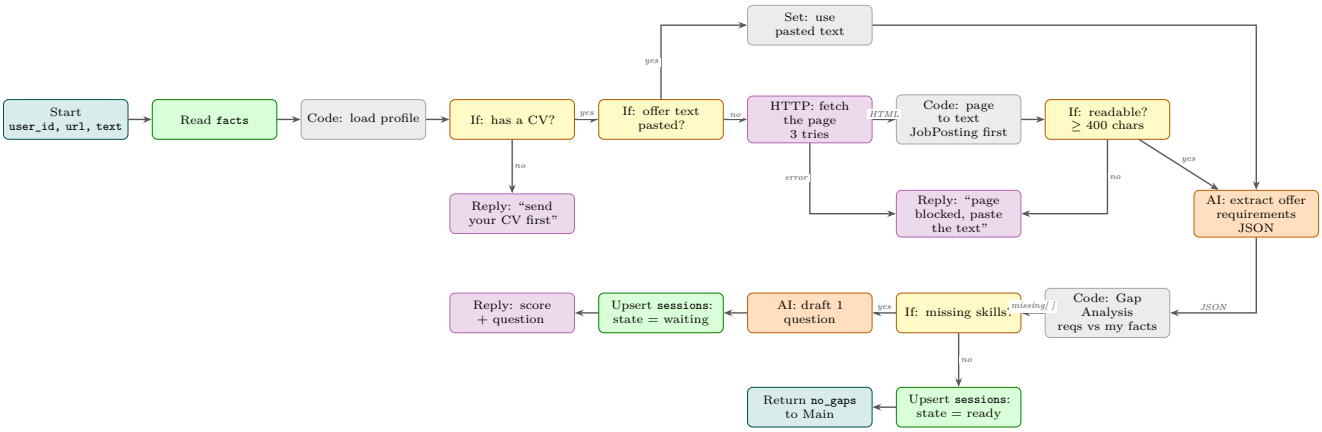


Figure 3: Analyzer. Every exit ends in a reply to the user or a status returned to Main.

fails or the text is under 400 characters (a JavaScript-only page), the bot asks for `/job text:` instead of guessing.

3.3 3. Generator: writing the documents

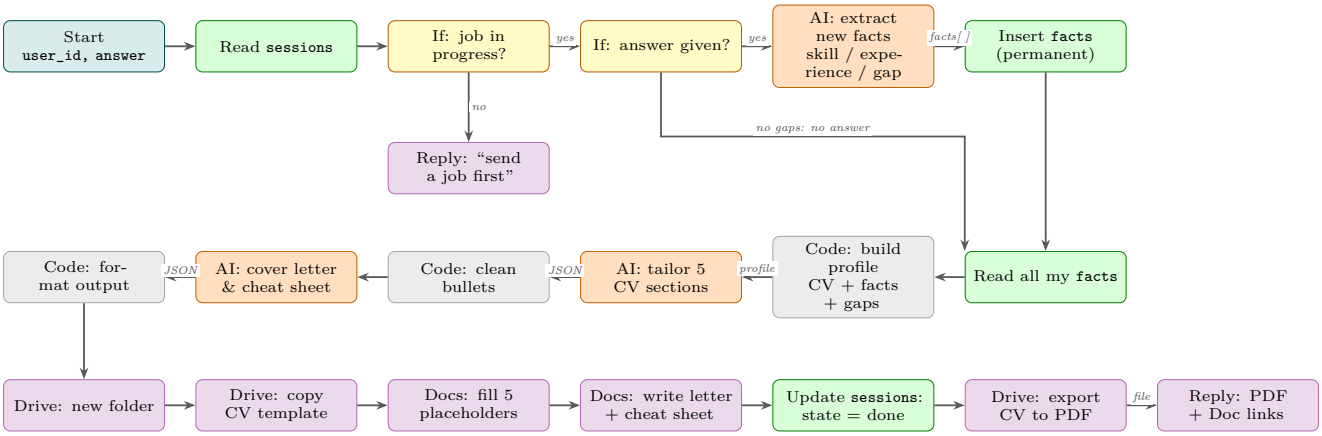


Figure 4: Generator. Three AI calls, then publishing to Google and back to Discord.

The CV is a template, not free text. My real CV lives in Google Docs with five placeholders: `{{SUMMARY}}`, `{{DIGIMOBEE_BULLETS}}`, `{{ENEDIS_BULLETS}}`, `{{ESSEC_ECON_BULLETS}}`, `{{SKILLS}}`. The AI only writes those five fields; dates, titles, schools and layout are never touched.

Why: an AI that rewrites a whole CV breaks the layout and can change facts like dates. Filling five fields keeps the CV recognisably mine and always one page.

Against hallucination. The profile sent to the AI is the base CV, then "facts confirmed by the candidate", then "known gaps" (things I said I don't have), then "job requirements not found on the CV". The prompts forbid inventing skills, tools or numbers and allow a requirement only if a confirmed fact covers it. The CV prompt keeps each bullet about its own experience. The letter and cheat-sheet prompt has a "strict facts" rule: no invented motivations, methods or results; when a STAR story needs a detail the profile does not give, the cheat sheet writes `[add your detail]` instead of inventing one. Temperatures are low for extraction (0.1) and higher for the letter (0.8).

3.4 4. Error handling: three layers

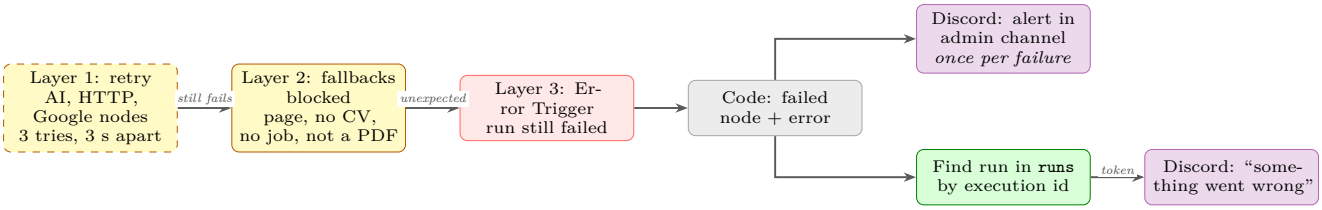


Figure 5: Error handling. Expected problems get a friendly fallback; unexpected ones reach the Error Handler.

The Error Handler is set as the error workflow of Main, Analyzer and Generator. The hard part is telling the *user*:

the error trigger only knows the failed execution, not who sent the command. That is why Main writes a `runs` row (execution id → user, reply token) at the start of every command. The admin alert is set to "continue on error", so the user is warned even if the alert itself fails. When a sub-workflow fails, Main fails too (on its Run Analyzer or Run Generator node), so n8n starts the Error Handler twice. An If node, Not alerted yet?, skips the second admin alert; the user is still warned exactly once, through Main's `runs` row.

4 Data model

Table	Columns	Role
`facts`	`user_id, kind, value, source, updated_at`	Long-term memory. `kind` is `base_cv` (full CV text), `cv_profile` (parsed JSON), `skill`, `experience`, `gap` or `answer`. One flexible table instead of one column per skill.
`sessions`	`user_id, state, job_url, job_json, job_text, missing_json, question, match_score, updated_at`	Short-term memory: the job in progress. One row per user; `state` goes `waiting` → `done`, or `ready` → `done` when nothing is missing. A `/answer` on a `done` session is refused ("no job in progress"), so an old job is never regenerated by mistake.
`runs`	`execution_id, user_id, app_id, token, command, created_at`	Lets the Error Handler reply to the right user.

Why: the conversation spans several Discord commands, each one a separate n8n execution. Without `sessions`, `/answer` would not know which job it answers. Without `facts`, the bot would forget every skill I confirm.

5 Minimum technical requirements, where each one is

Requirement	Node(s) in the build
If or Set node	If: Signature valid?, Has CV?, Readable?, Missing skills?, No gaps?. Set: the payloads passed to sub-workflows.
Code node	Verify & Parse (signature), Clean Page to Text, Code: Gap Analysis, Build Profile, Format Error.
Router or Switch	Interaction Type and Command Router (`/cv` / `/job` / `/answer`).
API connection	HTTP Request to job pages and Discord API; Google Drive and Docs; OpenRouter.
Message trigger	Discord slash commands through the Interactions webhook.
Database	n8n Data Tables `facts`, `sessions`, `runs`.
AI node	6 LLM Chains on Gemini (5 with a structured JSON output parser): parse CV, extract offer, draft question, extract new facts, tailor CV, letter + cheat sheet.
Sub-workflow	Analyzer and Generator, called through Execute Workflow; the Generator is reused from two places.
Error handling and retries	Retry on fail (3 tries) on AI, HTTP and Google nodes; fallback branches; Error Workflow for 3 workflows.

Why this is Level 3. Several stages, conversational state across separate executions, two sub-workflows, three services chained (Discord, OpenRouter, Google), and robustness as part of the design: signature check, retries, fallbacks and an error workflow.

6 Test results

The fixed test set from the scoping was first run on 21 October, then **re-run end to end on the final build on 22 October**, through Discord, with a real offer: Ledger, Data Analyst internship, on Welcome to the Jungle. All 12 executions of the final run succeeded in n8n.

Test input	What happened	Result
PDF upload (`/cv`)	"CV saved for POLLUX GRONIER: 29 skills/tools, 6 experiences." A `.docx` sent by mistake (21 Oct) was refused with "attach a PDF".	Pass
Link to a posting (`/job url:`, Ledger)	Page read through its JobPosting block. First match 5/10: SQL, Python, statistical analysis and machine learning found; Master's degree, Generative AI and autonomy asked about in one question. All 4 missing items saved.	Pass
Answer (`/answer`)	New facts saved (gap: "not a Master's student"; skills: RAG, LangChain, Transformers; tutoring experience). Tailored CV as PDF in Discord, cover letter, cheat sheet and Drive folder in about one minute. Digimobee and Enedis bullets on the right job; Tableau and Snowflake not claimed.	Pass
Same offer, pasted (`/job text:`)	7/10 instead of 5/10: Generative AI is now matched because the bot remembered the previous answer.	Pass
`/answer` with no job in progress	"I have no job offer in progress. Send one first…"	Pass
Blocked or JavaScript-only page (JobTeaser, 21 Oct)	"I couldn't read that page… send it with `/job text:`"	Pass
`/help`	Usage message, instantly.	Pass

7 Problems encountered, and how I solved them

7.1 Platform difficulties

- **No Discord message trigger on n8n Cloud.** Reading messages needs a permanent Gateway connection. I switched to slash commands received by a webhook (section 1).
- **Signature and the 3-second rule.** Discord refused to save the endpoint URL until the Ed25519 check worked, and any reply slower than 3 seconds fails. Hence the Code node that verifies the signature and the immediate deferred "thinking…" answer.
- **Interaction tokens.** Every reply is an edit of the original message, and the token expires after 15 minutes, so a whole application must finish within that time (it takes about one minute).
- **Discord permissions.** Registering commands and posting admin alerts failed with "Missing Access" until the bot was re-invited with the commands scope and the View Channel permission.

7.2 Bugs found by the tests

Problem	Effect	Fix
Gap code read the "text pasted?" If node	Crash on every link: that branch is empty when a link is sent	Read the original input instead. The Error Handler caught the crash, which also tested layer 3.
Invented skills	First CV claimed SQL and Tableau, just flagged as missing	"Not found on the CV" list in the profile; skills line limited to CV and confirmed facts.
False matches	"Git" matched inside "github.com" and "BI tools" matched the JSON field name "tools": the Pigment offer scored 3/10 instead of 0/10	Whole-word matching on profile values only.
Only 3 missing items saved	The Generator could claim the 4th and later ones	Save all missing items; ask about the first three.
The saved CV was the template	No content for the Digimobee and Enedis bullets	Uploaded the real CV.
Designed CV PDF read in the wrong order	Digimobee bullets came out under the Enedis heading: the AI could swap internships	Upload a clean, text-ordered PDF with the same content.
Letter invented a motivation, cheat sheet invented details	"I've been following Ledger for a while"; made-up methods	"Strict facts" rule and `[add your detail]` placeholders.
Two admin alerts per failure	Noise in the admin channel	If node Not alerted yet? in the Error Handler.
`/answer` after a finished job	Regenerated the old job	Generator checks `state != done`.
Unknown interaction types	No reply at all	Switch fallback output to the usage message.

8 Limits and next steps

- **Built for one person.** The CV template (my five placeholders) and the letter signature are mine. The data is already keyed by `user_id`; opening it to others means one template per user.
- **Keyword matching misses synonyms** ("PostgreSQL" vs "SQL"). Next step: let the AI return synonyms with each requirement, or use embeddings.
- **The bot trusts what you answer.** It never checks a claimed skill, and the same fact can be saved twice in different words. Harmless, but could be deduplicated.
- **One job at a time per user**: a new `/job` replaces the session in progress.
- **Input quality**: a multi-column designed CV must be sent as a clean, text-ordered PDF; JavaScript-only job pages need the pasted-text fallback (a headless browser would remove it).
- **Ops**: n8n Cloud trial; the bot token and API key must be rotated, and a `/forget` command added for privacy, before any real use.
- **From the scoping**: Kanban tracker, inbox watcher for recruiter replies, automated follow-ups. The `sessions` table already stores the job, so a tracker is a small addition.

9 Demo script (about 4 minutes)

1. `/help`: the instant answer (no AI, under 3 s).
2. Main canvas: signature check, deferred answer, `runs` log, Command Router. Show the `facts` table: the CV is already saved (29 skills/tools).
3. `/job url:` a new Welcome to the Jungle offer: score, found vs missing skills, one question. Show the `sessions` row in state `waiting`.
4. `/answer text:` a true answer: about one minute, explained on the Generator canvas meanwhile. Open the PDF and the letter from Discord; show the new rows in `facts`.
5. Error handling: retries on a node, the error workflow setting, an alert in the admin channel.
6. Backup if the live AI is slow: the 22 October Ledger run (executions and PDF) can be opened step by step.

10 Questions to prepare

What are the workflows and when is each one called?
Main runs on every slash command. It calls the Analyzer on `/job` and the Generator on `/answer`, or directly after the Analyzer when no skill is missing. The Error Handler is started by n8n when any of these three still fails. Setup runs once by hand.

Why slash commands and not a bot that reads every message?
Reading messages needs a permanent Gateway connection, which an n8n Cloud workflow cannot hold. Slash commands arrive as HTTP requests, so a webhook is enough. They are also structured: Discord already tells me "this is a file", "this is a URL". That removes a whole class of parsing bugs, and the user sees the expected arguments as they type.

How do you stay within Discord's 3-second limit when AI takes a minute?
Main answers immediately with a deferred response ("thinking…"), keeps running, and later edits that message with the interaction token.

How do you make sure only Discord can call your webhook?
Every request is signed with Ed25519. A Code node verifies the signature with the app's public key and returns 401 otherwise. Discord also tests this when the endpoint URL is saved.

Why sub-workflows?
Each one has a single job and can be tested on its own. The Generator is reused from two places. A failure is easy to locate because n8n records each sub-workflow as its own execution.

How does the bot remember the conversation?
Each command is a separate execution, so state lives in the database: `sessions` keeps the job in progress and the question asked, `facts` keeps everything learned about me, forever.

Why compute the gaps in a Code node and not with the AI?
The AI is good at reading messy text, so it extracts the requirements. Comparing two lists is deterministic, so code does it: free, instant, same result every time, and I can explain every decision.

How do you stop the AI from inventing things?
Four guards: only five template fields are generated; the profile marks confirmed facts, known gaps and unconfirmed requirements; the prompts forbid new skills, tools and numbers; and I tested for it. The tests did catch invented skills once, and the fix is in the build.

What happens if a step fails?
First it retries 3 times. Known problems (blocked page, no CV, wrong file, no job) have their own friendly reply. Anything else goes to the Error Handler, which alerts me with the failed node and tells the user.

Why n8n Data Tables and not Google Sheets or Postgres?
Built into n8n: no extra credential, typed columns, filters and upserts in a native node. The volume is tiny. Sheets would also work; Postgres would be overkill for this.

Why Gemini through OpenRouter?
Fast and cheap for five calls per application, good at structured JSON. OpenRouter means one key and a model swap in one field if a model is down or a better one comes out.

What would you do differently or add next?
Synonym-aware matching, several jobs in parallel, a headless browser for JavaScript pages, and the Kanban tracker from the scoping.

What did your tests find?
Ten real problems, from a crash on links to the AI inventing a motivation in the letter. The most surprising: the matching counted "Git" inside "github.com", so my first offer scored 3/10 instead of 0/10. All ten are fixed (section 7).

Why did the score go from 5/10 to 7/10 on the same offer?
Because of the memory: my answer about Generative AI was saved in `facts`, so the next analysis found it.

Can someone else use your bot?
Not yet: the CV template and the letter signature are mine. The data is already per user; I would store one template per user in `facts`.

What was the hardest part?
The Discord side, more than the AI: signature verification, the 3-second rule, interaction tokens, and permissions. Each one failed silently or with “Missing Access” until it was right. Second hardest: making the AI honest, which took four separate guards and real tests to catch what it invented.